# АНАЛИТИЧЕСКИЙ ДОКУМЕНТ ПО ПРОЕКТУ И ЕГО СВЯЗИ С ОЗЕРОМ ДАННЫХ

*для темы магистерской диссертации: «Разработка сервиса интеллектуального анализа данных для поддержки принятия решений в области пожарной безопасности сельских территорий»*

Подготовлено на основе изучения структуры и кода проекта в рабочем репозитории.

Дата подготовки: 21.04.2026

**Практический вывод документа: ваш проект логично позиционировать как интеллектуальный аналитический сервис и уровень поддержки решений, работающий поверх озера данных или lakehouse-платформы.**

## Оглавление

*Оглавление будет обновлено при открытии документа в Microsoft Word.*

## 1. Краткий итог исследования

После изучения кода можно сделать главный вывод: ваш проект уже вышел за рамки простой загрузки таблиц и фактически является предметно-ориентированной аналитической платформой для поддержки решений по пожарной безопасности сельских территорий.

- В проекте реализованы импорт и загрузка данных в PostgreSQL, профилирование признаков, интеллектуальная очистка, создание очищенных таблиц, веб-интерфейс на FastAPI, аналитический dashboard, прогнозирование, ML-модуль, кластеризация территорий, ранжирование проблемных точек, картографический блок и формирование кратких управленческих выводов.
- По состоянию на момент анализа в репозитории обнаружено 344 Python-файлов, из них 140 относятся к прикладным сервисам, 18 к dashboard-логике, 14 к pipeline-обработке и 18 к картографическому/геоаналитическому слою.
- В проекте выделяется не менее 22 API-эндпоинтов и 16 пользовательских web-маршрутов, а также 49 тестовых сценариев и файлов кейсов, что подтверждает зрелость кодовой базы по меркам магистерского проекта.
- С архитектурной точки зрения проект уже содержит элементы будущего lakehouse-контура: есть сырой импорт, этапы очистки и семантического отбора колонок, есть производные очищенные таблицы, есть материализованные представления для forecasting, а над ними построен интеллектуальный сервис принятия решений.
- Связка с темой партнера естественная: партнерский проект может быть основанием хранения, каталогизации, версионирования и выдачи данных, а ваш проект — прикладным аналитическим слоем, который преобразует эти данные в прогнозы, рейтинги рисков, карты и понятные рекомендации руководителю.

## 2. Что уже представляет собой ваш проект

Текущий репозиторий не выглядит как учебный шаблон. Это достаточно развитый сервис с web-слоем, отдельным pipeline-ядром и специализированными аналитическими подсистемами. Ниже приведены ключевые наблюдения по фактической реализации.

| Показатель | Значение | Комментарий |
|---|---|---|
| Язык и основная платформа | Python + FastAPI | Веб-интерфейс, API и аналитические вычисления. |
| Основная БД | PostgreSQL | Используется как текущее хранилище и источник аналитических запросов. |
| Форматы загрузки | XLS, XLSX, CSV | Импортируются через отдельный pipeline-шаг. |
| API-эндпоинты | 22 | Dashboard, forecasting, clustering, ML, access points, таблицы, загрузка и профилирование. |
| Пользовательские web-страницы | 16 | Отдельные страницы под dashboard, прогноз, ML, кластеризацию, карту и таблицы. |
| Тестовые файлы и кейсы | 49 | Наличие тестов усиливает научно-практическую ценность работы. |

Анализ кода показывает следующую архитектуру. Файл app/main.py поднимает FastAPI-приложение, подключает API и маршруты страниц, а на старте выполняет проверку соединения с БД и прогрев кэшей. Это означает, что сервис рассчитан не только на локальный разовый запуск, но и на постоянную web-эксплуатацию.

Слой core/processing выполняет роль конвейера обработки. В нем есть класс Pipeline, а также последовательность шагов ImportDataStep, FiresFeatureProfilingStep, KeepImportantColumnsStep, CreateCleanTableStep и CreateFireMapStep. Такой набор соответствует логике data engineering: сначала загрузка источника, затем оценка качества и структуры, затем доменная фильтрация признаков, затем формирование очищенного набора и, при необходимости, построение картографического представления.

Сервисный слой app/services уже ориентирован на прикладные сценарии принятия решений. Здесь есть модули прогнозирования, ML-модуль, кластеризация, ранжирование проблемных точек, прогноз риска, объяснимая логистика, краткие управленческие выводы и картографический сервис. Это особенно важно для вашей диссертации: вы можете аргументированно утверждать, что проект не просто хранит данные, а доводит их до уровня объяснимых и управленчески полезных выводов.

Ключевые функциональные блоки проекта можно описать так:

- Загрузка данных: import_data.py читает XLS/XLSX/CSV и сохраняет данные в PostgreSQL, а также экспортирует CSV-копию в результаты обработки.
- Профилирование: fires_feature_profiling.py оценивает долю пропусков, доминирующие значения, вариативность и другие признаки качества колонок.
- Сохранение важных доменных признаков: keep_important_columns.py использует правила сопоставления и Natasha-поиск, чтобы не удалить значимые для предметной области колонки только из-за формальных порогов профилирования.
- Формирование очищенной таблицы: create_clean_table.py создает таблицу вида clean_<имя_таблицы> и экспортирует очищенный Excel-файл.
- Dashboard-аналитика: app/dashboard и app/routes/api_dashboard.py обеспечивают сводный мониторинг по данным пожаров.
- Прогнозирование: forecasting-пакет строит временные ряды, подготавливает дневные агрегаты и использует materialized views для ускорения расчетов.
- ML-анализ: ml_model-пакет содержит обучение, бэктестинг, оценку моделей, интервальные прогнозы и выдачу результатов через API.
- Кластеризация: clustering-пакет реализует сегментацию территорий и оценку качества кластеров.
- Рейтинг проблемных точек: access_points-подсистема ранжирует проблемные зоны по набору факторов, связанных с частотой инцидентов, удаленностью и прикрытием.
- Прогноз риска и геоаналитика: forecast_risk, core/mapping и fire_map_service связывают пространственный анализ, приоритизацию территорий и картографическое представление.
- Исполнительные краткие выводы: executive_brief формирует короткий управленческий текст для руководителя или дежурной смены.
- Фоновые задачи и кэширование: job_store, job_support и runtime caches показывают, что система рассчитана на тяжелые аналитические вычисления без блокировки интерфейса.

## 3. Сильные стороны текущей реализации

- Наличие четкого пайплайна обработки данных: это позволяет обосновать воспроизводимость вычислений.

- Разделение на core и app: есть отдельный слой переиспользуемой обработки и отдельный web/HTTP-слой.
- Наличие предметной логики, а не только общей аналитики: проект ориентирован именно на пожарную безопасность сельских территорий.
- Поддержка объяснимости: проект не ограничивается цифрами, а формирует текстовые пояснения и причины приоритета.
- Поддержка картографического и геоаналитического контура: это критично для пространственных задач сельских территорий.
- Наличие материализованных представлений для forecasting и индексов под них: это признак инженерной зрелости, а не только исследовательской идеи.
- Наличие тестов: это усиливает доверие к результатам и делает проект более серьезным с точки зрения диссертации.

## 4. Ограничения текущей архитектуры и почему возникает потребность в озере данных

Несмотря на сильную текущую реализацию, архитектура проекта остается в основном database-centric: PostgreSQL одновременно выступает местом хранения импортированных данных, источником аналитики и площадкой для производных представлений. Для одной команды и одного типа табличных наборов это рабочий вариант, но для более серьезной межведомственной, многолетней и многомодельной системы пожарной безопасности сельских территорий этого уже недостаточно.

- Нет выделенного постоянного raw-слоя, где хранились бы все исходные данные в неизменном виде с историей версий.
- Нет универсального хранилища для разнородных форматов: таблиц, геоданных, логов, снимков, метеорологических рядов, документов и изображений.
- Нет отдельного единого каталога метаданных и управления происхождением данных на уровне всей экосистемы.
- Нет полноценного разделения между зонами raw, cleaned, curated, feature, serving.
- Нет механизма, который одинаково хорошо обслуживал бы и batch-интеграцию, и потоковые данные, и межсистемный обмен с внешними участниками.
- Для роста числа источников и сроков хранения PostgreSQL как единственная опора может стать узким местом по стоимости сопровождения и по гибкости схем.

Именно здесь возникает логичная роль озера данных. Оно не заменяет ваш сервис, а создаёт вокруг него более зрелую инфраструктурную основу.

## 5. Что такое озеро данных

Озеро данных — это централизованное хранилище, в котором можно сохранять структурированные, полуструктурированные и неструктурированные данные в исходном или близком к исходному виде, а затем использовать их для разных видов аналитики: SQL-запросов, BI-дашбордов, статистики, машинного обучения, пространственного анализа и построения цифровых сервисов.

Ключевая идея озера данных заключается в том, что данные сначала сохраняются, а схема их интерпретации может определяться позже. Такой подход особенно полезен в исследовательских и междисциплинарных проектах, где заранее невозможно точно предсказать все будущие вопросы к данным.

Для вашей предметной области это особенно уместно, потому что система пожарной безопасности сельских территорий включает в себя:

- реестры пожаров и инцидентов;
- метеорологические ряды;
- геопространственные слои: населенные пункты, границы, дорожная сеть, водоисточники, пожарные части, зоны доезда;
- данные о силах и средствах реагирования;
- исторические профили рисков по районам;
- неструктурированные документы и отчеты;
- в перспективе данные сенсоров, спутниковые снимки и лог-файлы интеграционных систем.

## 6. Чем озеро данных отличается от базы данных, хранилища данных и lakehouse

| Подход | Главная идея | Сильная сторона | Ограничение |
|---|---|---|---|
| Обычная БД | Хранить операционные таблицы по фиксированной схеме | Транзакционность и простота приложений | Плохо подходит для больших массивов разнородных и исторических данных |
| Data warehouse | Хранить уже очищенные и согласованные аналитические данные | Быстрые отчеты и BI | Менее гибок для сырых и неструктурированных данных |
| Data lake | Хранить почти все данные в исходном или слабопреобразованном виде | Гибкость, масштабируемость, пригодность для ML и исследований | Без каталога и правил быстро превращается в data swamp |
| Lakehouse | Совместить гибкость data lake и управляемость warehouse | ACID-таблицы, версионирование, SQL и ML поверх озера | Чуть сложнее в настройке, но на практике часто является лучшим вариантом |

Для диссертационного текста можно использовать такую формулировку: теоретически партнер разрабатывает озеро данных, а для достойной практической реализации целесообразно строить lakehouse, то есть озеро данных с табличными форматами, управлением схемой, историей версий и удобным SQL-доступом.

## 7. Почему тема вашего партнера и ваша тема хорошо связаны

Темы не просто совместимы, а образуют естественную двухуровневую архитектуру. Партнерский проект отвечает на вопрос «где и как хранить, каталогизировать и готовить данные», а ваш проект отвечает на вопрос «как на основе этих данных получить интеллектуальные выводы для принятия решений».

| Область | Проект партнера: озеро данных | Ваш проект: интеллектуальный сервис |
|---|---|---|
| Сбор данных | Прием данных из разных источников и форматов | Использует уже собранные данные |
| Хранение | Raw/Bronze, Silver, Gold, объектное хранилище, история версий | Использует curated-слои и быстрые витрины чтения |
| Каталог и метаданные | Описание наборов, происхождение, владельцы, качество | Использует каталог для поиска и выбора источников |
| Качество данных | Базовые правила валидации, контроль загрузок | Глубокое предметное профилирование и отбор |

| | | |
|---|---|---|
| | | значимых признаков |
| Аналитика | Подготовка витрин и слоев потребления | Прогнозирование, ML, кластеризация, риск-скоринг, карты и краткие управленческие выводы |
| Пользовательский интерфейс | Обычно не обязателен | FastAPI, web-страницы, API, отчеты |
| Итоговая ценность | Надежный фундамент данных | Поддержка принятия решений в реальной предметной области |

С научной точки зрения совместный вклад можно подать так: партнер создает инфраструктурный слой интеграции и хранения данных, а вы разрабатываете интеллектуальный прикладной слой, который превращает данные озера в управленческие решения в области пожарной безопасности сельских территорий.

## 8. Как связать именно ваш текущий код с будущим озером данных

Очень важно не противопоставлять существующий сервис озеру данных. Правильнее показать, что ваш проект уже содержит прикладной конвейер над данными, а озеро данных расширяет и усиливает этот конвейер.

| **Текущий модуль/этап** | **Что делает сейчас** | **Как это соотносится с озером данных** |
|---|---|---|
| ImportDataStep | Импортирует XLS/XLSX/CSV и пишет таблицу в PostgreSQL | Это локальный вариант слоя приема данных; в lakehouse он должен писать raw-артефакты в объектное хранилище и регистрировать метаданные |
| FiresFeatureProfilingStep | Проверяет пропуски, доминирование, вариативность колонок | Это часть data profiling и контроля качества в Silver-слое |
| KeepImportantColumnsStep | Сохраняет доменно важные признаки | Это предметно-ориентированная семантическая очистка и защита признаков от потери |
| CreateCleanTableStep | Создает clean_таблицу и экспорт | Это аналог refined/silver-слоя или промежуточной curated-зоны |
| prepare_forecasting_aggregates | Готовит materialized views для forecasting | Это оптимизированный для чтения gold-слой или витрина потребления |
| forecasting / ml_model / clustering | Выполняют аналитические и предиктивные вычисления | Это интеллектуальный слой потребления данных поверх curated-слоев |
| forecast_risk / fire_map_service | Строят риск-профили и карты | Это уровень decision intelligence и geospatial consumption |
| executive_brief | Переводит аналитику в краткий управленческий текст | Это последний прикладной слой поддержки принятия решений |

Иными словами, ваш код уже соответствует верхним уровням архитектуры. Не хватает прежде всего полноценного инфраструктурного нижнего уровня, который и должен быть реализован в партнерской работе.

## 9. Рекомендуемая целевая архитектура на достойном уровне

Если делать озеро данных «на достойном уровне», то оно не должно ограничиваться папкой с CSV-файлами. Нужна полноценная многоуровневая архитектура, где есть хранение,

каталог, контроль качества, версии наборов, вычислительные механизмы и слой выдачи данных прикладным сервисам.

> Источники -> Ingestion -> Raw/Bronze -> Validation/Normalization -> Silver -> Gold/Feature/Serving -> FastAPI сервис -> Дашборды, карты, прогнозы, ML, рекомендации

Ниже приведен практический состав такой архитектуры.

1. Источники данных. Реестры пожаров, выгрузки ведомственных систем, погодные данные, геоданные, дорожная сеть, водоисточники, сведения о пожарных частях, демография, нормативные и отчетные документы, а в перспективе сенсорные и потоковые данные.
2. Слой ingestion. Нужны регулярные загрузчики, которые принимают файлы, API-ответы, БД-реплики и события, снабжают данные метками времени и версий и сохраняют их в raw-зону без разрушения исходного содержания.
3. Raw/Bronze. Здесь данные лежат в максимально исходном виде. Этот слой нужен не для пользователя, а для воспроизводимости, аудита и возможности переобработки в будущем.
4. Silver. Здесь выполняются очистка, нормализация, дедупликация, преобразование дат и координат, унификация кодов районов, причин пожара и категорий объектов, а также первичная оценка качества.
5. Gold. На этом слое появляются витрины и агрегаты под конкретные задачи: прогноз по территориям, риск-скоринг, витрина response-time, витрина проблемных точек, daily aggregates, geospatial features и данные для executive dashboard.
6. Feature-слой для ML. Отдельно должны храниться наборы признаков и целевые переменные для воспроизводимого обучения и переобучения моделей.
7. Serving-слой. Ваш FastAPI-сервис может работать не напрямую с raw-слоем, а с curated-слоями и специализированными витринами потребления в PostgreSQL, Trino или другом SQL-движке.
8. Каталог и метаданные. Каждый набор должен иметь описание, владельца, назначение, период обновления, схему, показатели качества и происхождение.
9. Контроль качества. Для каждой загрузки нужны формализованные проверки: полнота, своевременность, допустимые диапазоны значений, уникальность, отсутствие критических пустых полей и согласованность кодов.
10. Оркестрация. Нужен планировщик, который запускает загрузки, пересчеты silver/gold-слоев, построение витрин, обучение моделей и публикацию метаданных.
11. Безопасность и разграничение доступа. Нужно разделять права для операторов загрузки, аналитиков, исследователей, разработчиков и руководителей, а также вести аудит обращений к данным.
12. Наблюдаемость. Для «достойного уровня» важно видеть, когда загрузка не пришла, какой набор просрочен, сколько данных отбраковано и какие витрины устарели.

## 10. Что обязательно должно быть в хорошем озере данных

- Разделение на слои raw, refined, curated и serving.
- Объектное хранилище или другой масштабируемый слой хранения, а не только одна реляционная БД.
- Поддержка форматов Parquet и табличного формата типа Iceberg или Delta для больших аналитических данных.
- Каталог метаданных и data lineage.
- Контроль качества данных на каждой загрузке.
- Версионирование и возможность воспроизведения состояния набора на нужную дату.
- Понятные правила именования и описания наборов.
- Регулярное обновление витрин, а не только ручной импорт.

- Интеграция с аналитическим сервисом, BI и ML-контурами.
- Механизмы архивации, резервного копирования и аудита.

Если хотя бы половины этих пунктов нет, то на защите могут сказать, что это не озеро данных, а просто файловое хранилище или набор разрозненных выгрузок.

## 11. Методы реализации озера данных

С методической точки зрения реализация озера данных должна опираться не только на инструменты, но и на правильные методы работы с данными.

| **Метод** | **Суть метода** | **Зачем нужен в вашей предметной области** |
|---|---|---|
| Schema-on-read | Схема интерпретации может применяться после загрузки | Позволяет хранить разнородные источники и не терять данные из-за жесткой схемы на входе |
| ELT/ETL | Данные сначала извлекаются и загружаются, затем преобразуются | Удобно отделять raw-слой от аналитических витрин и повторять обработку |
| Data profiling | Измерение качества, заполненности, вариативности и структуры данных | У вас уже реализован этот подход в профилировании признаков |
| Data quality validation | Формальные правила качества для каждой загрузки | Снижает риск построения ложных прогнозов и неверных управленческих выводов |
| Deduplication and harmonization | Удаление дублей и согласование справочников | Критично при объединении данных из разных районов и ведомств |
| Feature engineering | Построение производных признаков для моделей | Нужно для прогнозирования пожаров, логистических рисков и ранжирования территорий |
| Time-series forecasting | Построение прогноза по временным рядам | Непосредственно связано с вашим forecasting и ML-блоком |
| Clustering | Группировка территорий по сходству профилей | Позволяет выявлять типовые группы сельских территорий и схожие риски |
| Risk scoring | Расчет интегральной оценки приоритета | Нужен для поддержки управленческих решений и ранжирования районов |
| Explainable analytics | Подача результата с объяснением причин | В вашем проекте это особенно ценно из-за кратких управленческих выводов и объяснимой логистики |
| Geospatial analytics | Пространственный анализ, зоны доезда, hotspots, карты | Для сельских территорий пространственный аспект принципиален |

## 12. Рекомендуемые средства и технологии

Ниже приведен практический стек, который можно обосновать как современный и достойный для магистерской работы. Он подобран так, чтобы сохранялась научная логика, инженерная реалистичность и связь с вашим существующим Python/FastAPI-проектом.

| **Компонент** | **Что рекомендуется** | **Почему это разумно** |
|---|---|---|
| Хранение объектов | S3-совместимое объектное хранилище, например MinIO | Подходит для raw-слоя, Parquet, геоданных, логов, |

| | | |
|---|---|---|
| | или облачный S3 | отчетов и исторических версий |
| Формат аналитических файлов | Parquet | Колончатый формат хорошо подходит для аналитики и уменьшает объем хранения |
| Открытый табличный формат | Apache Iceberg как основной вариант; Delta Lake как сильная альтернатива | Дает ACID, метаданные таблиц, эволюцию схемы и time travel |
| Тяжелые batch-преобразования | Apache Spark | Подходит для крупных преобразований, расчета витрин и будущего масштабирования |
| Локальная/исследовательская аналитика | DuckDB | Очень удобен для локального анализа Parquet и быстрых прототипов без тяжелого кластера |
| Оркестрация | Apache Airflow | Позволяет по расписанию запускать ingestion, quality checks, пересчет слоев и переобучение моделей |
| SQL-доступ к озеру | Trino | Дает единый SQL-доступ к данным в объектном хранилище и другим источникам |
| Витрина потребления для приложения | PostgreSQL | Ваш текущий сервис уже хорошо интегрирован с PostgreSQL, поэтому его разумно сохранить как слой потребления |
| Каталог и метаданные | OpenMetadata | Нужен для поиска наборов, lineage, качества, владельцев и управления данными |
| Качество данных | Great Expectations (GX Core) | Позволяет формализовать проверки качества и документировать результаты валидации |
| ML lifecycle | MLflow | Подходит для отслеживания экспериментов, версий моделей, метрик и артефактов обучения |
| Прикладной сервис | FastAPI + существующая кодовая база | Позволяет не выбрасывать текущую разработку, а встроить ее в зрелую инфраструктуру |

Если ресурсов мало и вы делаете первую устойчивую версию, то возможен компромиссный вариант: MinIO + Parquet + PostgreSQL + Airflow + Great Expectations + MLflow + ваш FastAPI-сервис. Если же вы хотите показать более зрелую архитектуру, лучше писать про lakehouse-вариант: MinIO + Iceberg + Spark + Trino + PostgreSQL + OpenMetadata + Great Expectations + MLflow + ваш сервис.

## 13. Практический сценарий внедрения для вашей пары проектов

13. Шаг 1. Согласовать перечень источников данных: реестры пожаров, погода, геоданные, силы реагирования, населенные пункты, дороги, водоисточники, при необходимости фото и документы.
14. Шаг 2. Описать единый реестр наборов данных: название, владелец, периодичность обновления, формат, критичность, набор обязательных полей.

15. Шаг 3. Развернуть объектное хранилище и организовать в нем зоны raw, silver, gold и artifacts.
16. Шаг 4. Реализовать загрузчики, которые складывают исходные наборы в raw, не разрушая оригинальную структуру и сохраняя дату загрузки.
17. Шаг 5. Реализовать silver-процессы: очистка, нормализация дат, районов, причин пожара, категорий объектов, координат и единиц измерения.
18. Шаг 6. Перенести вашу логику профилирования и сохранения важных колонок в quality/semantic-layer над silver-слоем.
19. Шаг 7. Создать gold-витрины: ежедневные агрегаты для forecasting, витрину по территориям, витрину по response time, витрину по access points, geospatial features и ML features.
20. Шаг 8. Настроить публикацию curated-наборов в витрину потребления, с которой работает ваш FastAPI-сервис.
21. Шаг 9. Переключить dashboard, прогнозирование, ML и риск-скоринг на чтение из curated-слоев и витрин потребления вместо прямой зависимости от ручного импорта.
22. Шаг 10. Подключить метаданные, lineage и data quality dashboards.
23. Шаг 11. Настроить переобучение моделей и журналирование экспериментов в MLflow.
24. Шаг 12. Описать регламент эксплуатации: как загружаются данные, кто отвечает за качество, когда обновляются витрины, как проверяется корректность прогнозов.

## 14. Как оставить ваш текущий PostgreSQL и при этом построить озеро данных

Очень важный практический тезис: не нужно отказываться от PostgreSQL. В зрелой архитектуре PostgreSQL может остаться serving-слоем, то есть местом, откуда FastAPI быстро читает уже подготовленные таблицы, materialized views и витрины. При этом само озеро данных хранит полную историю, сырой слой, файлы большого объема и промежуточные аналитические наборы.

- Озеро данных хранит всю историю и разнородные форматы.
- Silver и Gold формируются в lakehouse-слоях и при необходимости частично публикуются в PostgreSQL.
- Ваш текущий FastAPI-сервис продолжает работать с PostgreSQL, но получает более качественные и более богатые источники.
- Таким образом, текущая кодовая база не ломается, а масштабируется архитектурно.

## 15. Как можно сформулировать связь тем в тексте диссертации

Ниже приведены готовые формулировки, которые можно адаптировать под введение, постановку задачи или раздел об архитектуре.

- Озеро данных рассматривается как инфраструктурная основа консолидации, хранения и подготовки разнородных данных о пожарной обстановке, сельских территориях, силах реагирования и внешних факторах риска.
- Разрабатываемый сервис интеллектуального анализа данных использует curated-слои озера данных для построения аналитических витрин, прогнозов, риск-оценок, картографических представлений и рекомендаций, поддерживающих принятие решений в области пожарной безопасности сельских территорий.
- Таким образом, проект партнера обеспечивает управление данными, а ваш проект обеспечивает интеллектуальное использование данных в прикладном контуре принятия решений.
- Совместно проекты образуют единую архитектуру: от интеграции и хранения данных до объяснимой аналитики и пользовательского интерфейса для руководителей и специалистов.

Еще одна удачная формулировка: «В работе предлагается рассматривать озеро данных как базовый слой хранения и подготовки интегрированных данных, а интеллектуальный сервис как верхний слой аналитического и предиктивного преобразования данных в решения, ориентированные на повышение пожарной безопасности сельских территорий».

## 16. На что сделать акцент при защите

- Ваш проект уже содержит реальную аналитическую функциональность, а не только теоретическую схему.
- Вы не дублируете тему партнера, а надстраиваетесь над ней логически и архитектурно.
- Озеро данных оправдано предметно: слишком много разных типов данных и слишком высокая ценность истории и повторной обработки.
- Ваш сервис решает конечную прикладную задачу руководителя: кого считать приоритетом, где усилить контроль, какие территории несут повышенный риск и как это обосновать.
- Совместная архитектура хорошо масштабируется: можно добавлять новые источники без радикальной переделки прикладного сервиса.

## 17. Типичные ошибки при реализации озера данных

- Считать озером данных обычную папку с CSV-файлами без каталога и правил.
- Не хранить raw-версию данных и сразу перезаписывать исходники преобразованными файлами.
- Не фиксировать происхождение данных и дату загрузки.
- Не делать quality checks и рассчитывать аналитику на непроверенных данных.
- Смешивать сырые, очищенные и отчетные данные в одном и том же каталоге или одной таблице.
- Не отделять витрины для приложения от базового слоя хранения.
- Не учитывать геоданные и внешние факторы риска, ограничиваясь только реестром инцидентов.

## 18. Рекомендуемая дорожная карта по срокам

| Этап | Содержание | Результат |
|---|---|---|
| Этап 1 | Инвентаризация источников и проектирование метаданных | Каталог источников и схема предметной области |
| Этап 2 | Развертывание хранения и raw-слоя | Работающая загрузка исходных данных |
| Этап 3 | Silver-процессы и контроль качества | Нормализованные и проверенные наборы |
| Этап 4 | Gold-витрины и feature-слой | Подготовленные наборы для dashboard, forecasting, ML и risk-scoring |
| Этап 5 | Интеграция с вашим FastAPI-сервисом | Работающее приложение поверх curated-данных |
| Этап 6 | MLflow, мониторинг и эксплуатационные регламенты | Повышение зрелости и воспроизводимости решения |

## 19. Итоговое позиционирование вашего проекта

Самое точное позиционирование вашего проекта звучит так: это сервис интеллектуального анализа данных и поддержки принятия решений, который использует данные пожарной безопасности сельских территорий для построения аналитических, прогностических,

кластеризационных и картографических представлений, а также для формирования объяснимых управленческих рекомендаций.

В этой формулировке озеро данных выступает как фундамент хранения и подготовки данных, а ваш сервис — как прикладной механизм интеллектуального использования этих данных. Такое разделение ролей выглядит логично, современно и методически сильно для защиты магистерской диссертации.

## 20. Рекомендуемые источники и официальные материалы

Ниже приведены официальные источники, на которые можно опираться при описании современной реализации озера данных и lakehouse-подхода. Ссылки подобраны по состоянию на 21.04.2026.

- AWS. What is a Data Lake: https://aws.amazon.com/big-data/data-lake-on-aws/
- AWS. What is a Data Lakehouse: https://aws.amazon.com/what-is/data-lakehouse/
- Apache Iceberg Documentation: https://iceberg.apache.org/docs/latest/
- Delta Lake Documentation: https://docs.delta.io/
- Apache Spark Documentation: https://spark.apache.org/docs/latest/
- Apache Airflow Documentation: https://airflow.apache.org/docs/index.html
- Trino Official Site and Docs: https://trino.io/
- DuckDB Official Site: https://duckdb.org/
- OpenMetadata Documentation: https://docs.open-metadata.org/
- Great Expectations Documentation: https://docs.greatexpectations.io/
- MLflow Documentation: https://mlflow.org/docs/latest/
- Apache Kafka Introduction: https://kafka.apache.org/intro

Для текста диссертации лучше всего опираться на комбинацию: одно общее определение data lake/data lakehouse, одно описание открытого табличного формата, одно описание оркестрации, одно описание каталога метаданных, одно описание контроля качества и одно описание ML lifecycle.

## 21. Заключение

Изучение кода проекта показывает, что ваша работа уже имеет серьезное прикладное содержание и инженерную основу. Сервис реализует полный цикл интеллектуальной аналитики над данными о пожарах: от загрузки и профилирования до прогнозов, кластеризации, геоаналитики и формулирования коротких объяснимых рекомендаций.

Связь с озером данных является не искусственной, а естественной. Для такой предметной области требуется хранить большие разнородные и исторические массивы данных, поддерживать повторную обработку, вести контроль качества и выдавать curated-наборы в прикладной сервис. Следовательно, проект партнера может выступать инфраструктурной базой, а ваш проект — интеллектуальным слоем поддержки принятия решений.

Если на защите вы покажете именно такую архитектурную связку, то обе темы будут выглядеть не конкурирующими, а взаимодополняющими и образующими единое законченное решение.

## Приложение. Обнаруженные API-маршруты проекта

Список API-маршрутов сформирован автоматически по коду app/routes/api*.py и может использоваться в описании прикладной зрелости проекта.

- /api/access-points-data
- /api/clustering-data
- /api/clustering-jobs
- /api/clustering-jobs/{job_id}
- /api/column-search
- /api/column-search/create-modify-table
- /api/column-search/preview
- /api/dashboard-data
- /api/forecasting-data
- /api/forecasting-decision-support-jobs
- /api/forecasting-decision-support-jobs/{job_id}
- /api/forecasting-metadata
- /api/ml-model-data
- /api/ml-model-jobs
- /api/ml-model-jobs/{job_id}
- /api/tables/delete
- /api/tables/{table_name}
- /api/tables/{table_name}/page
- /import_data
- /logs
- /run_profiling
- /upload

## Приложение. Пользовательские маршруты интерфейса

- /
- /access-points
- /assets/plotly.js
- /backtesting
- /brief/dashboard.txt
- /brief/forecasting.txt
- /clustering
- /column-search
- /favicon.ico
- /fire-map
- /fire-map/embed
- /forecasting
- /ml-model
- /select_table
- /tables
- /tables/{table_name}

## Приложение. Текущие ключевые зависимости проекта

- fastapi>=0.115
- uvicorn>=0.30

- jinja2>=3.1
- python-multipart>=0.0.9
- SQLAlchemy>=2.0
- psycopg2-binary>=2.9
- numpy>=1.26
- pandas>=2.2
- plotly>=5.24
- scikit-learn>=1.5
- statsmodels>=0.14
- joblib>=1.4
- openpyxl>=3.1
- natasha>=1.6